

해온카드 시연 런북 v1.0

- 날짜 2026-09-18 · 브랜치 feat/haeon-lab-baseline
- 주소 북웨이브 localhost:8080 · 해온카드 localhost:8085 · 마이페이지 localhost:8085/mypage
- 계정 haeon01 / Haeon!2026 (화면에 띄우지 말 것)
- 모든 명령은 WSL 셸에서 프로젝트 루트. Windows PowerShell에는 docker 가 없다.

이 문서만 보고 처음부터 끝까지 돌릴 수 있게 썼다. 각 단계는 ① 무엇을 보여주나 → ② 명령 → ③ 기대 결과 → ④ 말할 문장 순서다.

한눈에 보기

	단계	시간	필수
A	준비 — 기동·기준선	5분	필수
B	화면 — 북웨이브 서점 · 해온카드 마이페이지	5분	필수
C	결제 — 승인·역등·거절	6분	필수
D	CARD-03 — 한도 이중 사용 Before/After	10분	핵심
E	가상 웹셀 기반 결제서버 침입 재현	5분	핵심
F	네트워크 격리 점검	2분	선택
G	정리 — 기준선 복원	3분	필수

북웨이브 쪽 준비가 안 됐다면 **부록 B. 해온카드 단독 시연**으로 카드사 몫만 따로 돌릴 수 있다. 영상으로 찍는다면 **부록 C. 촬영 가이드**를 먼저 읽는다 — 장비 구성, 화면 배치, 터미널이 필요한 구간, D단계 진행 순서가 정리돼 있다.

촬영 순서 규칙 — B → C 를 먼저 찍고, E와 D를 찍은 뒤 D를 마지막 장면으로 둔다. D는 카드 한도를 100,000 → 10,000,000으로 바꾸고 이용내역을 전부 지운다. D를 먼저 찍으면 B·C 화면의 숫자가 달라진다. 발표 재생은 E → D를 핵심 서사로 두고, D 영상은 **D-3 Before** 상태에서 시작한다. 시간이 모자라면 F를 빼고 E와 D는 끝까지 간다.

A. 준비 (5분)

A-1. 기동

```
cd /path/to/bookwave-haeon-lab
cp -n .env.example .env 2>/dev/null || true
docker compose --env-file .env config >/dev/null # 먼저 성공해야 한다
docker compose --env-file .env up -d
docker compose --env-file .env ps
```

기대 — 8개 컨테이너가 전부 **healthy**.

A-2. 포트 확인

```
curl -s http://localhost:8080/actuator/health; echo
curl -s http://localhost:8083/actuator/health; echo
for u in http://localhost:8080/ http://localhost:8085/ http://localhost:8085/mypage; do
  printf '%-40s ' "$u"; curl -s -o /dev/null -w '%{http_code}\n' "$u"
done
printf '%-40s ' '8085의 승인 API (차단되어야 함)'
curl -s -o /dev/null -w '%{http_code}\n' -X POST http://localhost:8085/internal/v1/authorizations
```

기대

항목	값
bookwave / mock-pg health	{"status": "UP"}
8080 / · 8085 / · 8085 /mypage	200
8085의 /internal/v1/authorizations	404

말할 문장: "카드사 서버는 포트를 두 개 쓴다. 회원이 여는 8085에는 가맹점 승인 API가 아예 열리지 않는다. 승인은 내부망 8084에서만 받는다."

A-3. 포털 데이터 (기존 볼륨에서 최초 1회만)

```
for f in db/haeon-card/init/004_portal_schema.sql \
        db/haeon-card/fixtures/006_portal_seed.sql \
        db/haeon-card/fixtures/007_portal_history_seed.sql; do
    docker exec -i bookwave-haeon-lab-haeon-card-mysql-1 \
        sh -c 'mysql -uroot -proot_lab_only haeon_card' < "$f"
done
```

세 파일 모두 여러 번 실행해도 안전하다. 출력이 없으면 정상이다.

A-4. 기준선 복원 (매 시연 직전 항상)

```
docker exec -i bookwave-haeon-lab-haeon-card-mysql-1 \
    sh -c 'mysql -uroot -proot_lab_only haeon_card' \
    < db/haeon-card/fixtures/003_card03_reset.sql
docker exec -i bookwave-haeon-lab-haeon-card-mysql-1 \
    sh -c 'mysql -uroot -proot_lab_only haeon_card' \
    < db/haeon-card/fixtures/007_portal_history_seed.sql

docker exec -i bookwave-haeon-lab-haeon-card-mysql-1 \
    sh -c 'mysql -uroot -proot_lab_only -t haeon_card -e "
        select c.card_token, c.is_primary, l.limit_amount, l.used_amount
        from card_limits l join cards c on c.card_id = l.card_id order by c.card_token"'
```

기대 — 이 네 줄이 나와야 한다. **B 단계 화면의 숫자가 여기서 나온다.**

카드	주	한도	이용금액
card-token-lab-001 해온 플러스		100,000	0
card-token-lab-002 해온 데일리	★	3,000,000	220,000
card-token-lab-003 해온 트래블		2,000,000	1,280,000
card-token-lab-011 (다른 회원)	★	1,500,000	190,000

A-5. 화면 미리 점검 (권장)

```
bash tools/portal-ui-check.sh
```

실제 브라우저로 홈 → 마이페이지 → 로그인 → 조회 → 로그아웃을 돌린다. **PASS (35/35)** 가 나오면 화면 준비 끝. 캡처 4장이 `evidence/runs/PORTAL-UI-*/` 에 남는다.

화면을 고쳤다면 먼저 이미지를 다시 만든다. `docker compose --env-file .env build haeon-card bookwave-app && docker compose --env-file .env up -d haeon-card bookwave-app` 브라우저는 Ctrl+F5.

B. 화면 (5분)

B-1. 북웨이브 서점 — `http://localhost:8080/`

보여줄 것: 가맹점 화면. 도서를 고르면 주문서가 열린다.

1. 랜딩을 한 번 훑는다.
2. 아무 도서나 `구매하기` → 주문서에 도서명·주문번호·금액·결제 카드가 채워진다.
3. **아직 결제하지 않는다.** C 단계에서 실제로 보낸다. 주문서를 닫는다.

말할 문장: "북웨이브는 가맹점이다. 결제를 시작하는 곳은 여기 한 곳뿐이다."

B-2. 해온카드 홈 — `http://localhost:8085/`

보여줄 것: 카드사 홈에는 **개인 정보가 하나도 없다.**

1. 카드·혜택·금융 안내가 보이고, 우측 MY 패널에는 로그인 폼만 있다.
2. 상단 `MY` 메뉴를 클릭한다.

B-3. 마이페이지 — `http://localhost:8085/mypage`

보여줄 것: 조회 화면은 로그인한 회원만 본다.

1. 주소가 `/mypage` 로 바뀌고 **로그인 화면**이 뜬다. (아직 로그인 전)
2. 틀린 비밀번호를 한 번 넣어 본다 → `아이디 또는 비밀번호가 올바르지 않습니다.`
3. `haeon01` / `Haeon!2026` 으로 로그인한다.
4. 주소는 그대로인 채 조회 화면으로 바뀐다.

기대 화면

영역	값
회원 정보	김해운 · <code>HC-MEMBER-001</code> · 보유 카드 3장
한도 요약	총 한도 5,100,000원 · 총 이용 1,570,700원 · 총 잔여 3,529,300원
보유 카드	해온 데일리 <code>**** 0002</code> (주 이용) / 해온 플러스 <code>**** 0001</code> / 해온 트래블 <code>**** 0003</code>
최근 이용내역	16건 (승인 13 · 거절 3)

5. 빵부스러기의 **홈** 을 눌러 돌아간다. 로그인은 유지되지만 **홈에는 회원번호도 이용내역도 없다.**

말할 문장 ①: "조회 대상 회원은 요청 값이 아니라 로그인 세션에서 나온다. 다른 회원(**haeon02**)으로 로그인하면 카드 한 장과 그 카드 내역만 보인다."

말할 문장 ②: "카드 한도 행의 **version** 은 증거용으로 DB에만 두고 회원 화면에는 내보내지 않는다. 화면에는 이용한도·이용금액·잔여한도만 보인다."

C. 결제 (6분)

C-1. 승인과 먹등 — 화면으로

결제 카드는 **해온 플러스 (**** 0001)** 를 **고른다**. 한도가 100,000원이라 한 건만 결제해도 사용률 막대가 눈에 띄게 움직인다.

1. 북웨이브에서 **느리게 읽는 밤** → **구매하기** → **결제하기**
2. **결제 승인**. 승인번호(AUTH-HC-...)를 화면에서 한 번 읽어 준다.
3. **같은 주문 결과 재조회** 클릭 → **세 식별자가 그대로**. 주문내역도 한 줄 그대로.
4. 해온카드 마이페이지(localhost:8085/mypage)로 이동 → **새로고침**

기대

	해온 플러스 **** 0001
결제 전	이용 0원 · 잔여 100,000원 · 사용률 0%
결제 후	이용 16,800원 · 잔여 83,200원 · 사용률 17%

이용내역 맨 위에 북웨이브 · 16,800원 · 결제 승인 이 북웨이브에서 읽은 것과 같은 승인번호로 올라온다.

이 장면이 오늘 시연의 핵심 중 하나다. 결제는 북웨이브가 시작하고, 결과는 해온카드에 남고, 회원은 해온카드에서 확인한다.

C-2. 거절 — 화면으로

주문서 금액을 **500,000원**으로 올려 결제한다.

- 북웨이브 화면: 승인 거절 · 카드 이용한도를 초과했습니다
- 마이페이지 이용내역: 승인 거절 / 이용한도 초과

말할 문장: "거절은 오류가 아니다. HTTP는 200이고, 카드사가 내린 정상 판정이다."

C-3. 검증 경계 — 터미널로 (선택)

```
TS=$(date +%s)
```

```
# 1) 한도 초과 → 200 + DECLINED
```

```
curl -s -w '[HTTP %{http_code}]\n' -X POST http://localhost:8080/api/v1/payments \
  -H 'Content-Type: application/json' \
  -H "X-Correlation-Id: DEMO-OVER-$TS" -H "Idempotency-Key: DEMO-OVER-$TS" \
  -d "{\"correlationId\":\"DEMO-OVER-$TS\",\"orderNo\":\"DEMO-OVER-$TS\",\"merchantRequestId\":\"DEMO-OVER-$TS\",\"amount\":500000,\"currency\":\"KRW\",\"paymentMethodToken\":\"card-token-lab-001\"}"
```

```
# 2) 필수값 누락 → 400
```

```
curl -s -w '[HTTP %{http_code}]\n' -X POST http://localhost:8080/api/v1/payments \
  -H 'Content-Type: application/json' -d '{}'
```

```
# 3) 상한 초과(>1천만) → 400, retryable:false
```

```
curl -s -w '[HTTP %{http_code}]\n' -X POST http://localhost:8080/api/v1/payments \
  -H 'Content-Type: application/json' \
  -H "X-Correlation-Id: DEMO-MAX-$TS" -H "Idempotency-Key: DEMO-MAX-$TS" \
  -d "{\"correlationId\":\"DEMO-MAX-$TS\",\"orderNo\":\"DEMO-MAX-$TS\",\"merchantRequestId\":\"DEMO-MAX-$TS\",\"amount\":99999999,\"currency\":\"KRW\",\"paymentMethodToken\":\"card-token-lab-001\"}"
```

	HTTP	응답
한도 초과	200	DECLINED / LIMIT_EXCEEDED / 승인액 0
필수값 누락	400	INVALID_REQUEST / 필수 헤더가 없습니다.
상한 초과	400	INVALID_REQUEST / retryable:false

증거를 남기려면 `bash tools/payment-flow-contract-check.sh`.

C가 끝나면 카드 사용액이 올라가 있다. D 시작 전에 반드시 D-2 기준선을 복원한다.

D. CARD-03 한도 이중 사용 (10분, 핵심)

같은 카드에 **6,000,000원 결제 두 건을 동시에** 보낸다. 한도는 10,000,000원이다. 한 건만 승인돼야 정상인데, 잠금이 없으면 두 건 다 승인된다.

D-0. Before / After가 무엇인가 (시연 전 30초 설명)

두 개는 서로 다른 코드 버전이 아니다. 같은 빌드 안에 두 경로가 다 들어 있고, 프로파일로 어느 쪽을 쓸지 고른다. 갈리는 지점은 한 줄이다.

```
boolean before = beforeBarrier.enabled();
CardLimit cardLimit = (before
    ? cardLimitRepository.findSnapshot(card.cardId())    // 잠금 없이 읽기 ← Before
    : cardLimitRepository.findForUpdate(card.cardId())); // SELECT ... FOR UPDATE ← After
```

프로파일	한도 행 읽기	동시 요청 결과
<code>before</code> + <code>BEFORE_BARRIER_ENABLED=true</code>	잠금 없이 스냅샷	둘 다 같은 값을 읽고 둘 다 승인
<code>after</code>	행 잠금 후 최신 값으로 재판정	한 건만 승인
<code>normal</code> (평소 운영)	<code>after</code> 와 완전히 동일	한 건만 승인

세 가지만 기억하면 된다.

1. **"예전 커밋으로 되돌렸다"가 아니다.** 취약 경로를 켜는 프로파일로 다시 띄운 것이다. 오히려 이 편이 설명하기 좋다 — 같은 바이너리에서 **잠금 한 줄 차이로 결과가 갈린다.**
2. **`after` 는 특별한 모드가 아니다.** 평소 동작(`normal`)이 곧 After이고, 일부러 켜야 하는 예외가 Before다. `after` 라는 이름은 증거에 "개선 적용 후"라고 표시하려고 붙인 라벨이다.
3. **Before와 After 사이에 반드시 기준선을 복원한다.** Before가 끝나면 사용액이 12,000,000원(한도 초과)으로 남는다. 복원 없이 After를 돌리면 두 건 다 거절되어 "잘 막았다"가 아니라 "원래 못 쓰는 카드"처럼 보인다.

D-2 기준선 → D-3 Before → [기준선 복원] → D-4 After → D-5 판정

말할 문장: "지금부터 같은 서버를 프로파일만 바꿔 두 번 돌립니다. 먼저 한도 행을 잠그지 않는 경로로, 다음엔 잠그고 최신 값으로 다시 판정하는 경로로. 코드에서 갈리는 건 한 줄입니다."

D-1. 디버그 포트 열기 (필수)

재현 도구가 호스트에서 승인 API(8084)를 직접 호출해야 한다.

```
docker compose -f compose.yaml -f compose.debug.yaml --env-file .env \
  up -d --force-recreate haeon-card
sleep 20
curl -s http://127.0.0.1:8084/actuator/health; echo
```

말할 문장: "평소엔 8084가 닫혀 있고 결제대행사만 내부망으로 호출한다. 지금은 재현 도구가 승인 API를 직접 불러야 해서 잠깐 여는 것이고, 끝나면 닫는다."

포트를 열어도 경계는 유지된다(보여주면 좋다).

```
printf '8084의 회원 API : '; curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:8084/porta
l/v1/me/cards
printf '8085의 승인 API : '; curl -s -o /dev/null -w '%{http_code}\n' -X POST http://127.0.0.1:808
5/internal/v1/authorizations
```

둘 다 404.

D-2. 데모 기준선 (실행 직전 향상)

```
docker exec -i bookwave-haeon-lab-haeon-card-mysql-1 \
  sh -c 'mysql -uroot -proot_lab_only haeon_card' \
  < db/haeon-card/fixtures/004_card03_demo_reset.sql
```

기대 — 한도 10,000,000 / 이용 0 / version 0. (기존 볼륨에서 맨 처음 한 번만 005_card03_demo_amount_range.sql 도 실행한다.)

D-3. Before — 취약 재현

```
TS=$(date -u +%Y%m%dT%H%M%SZ); RUN=HC03-BEFORE-$TS; echo $RUN

LAB_PROFILE=before BEFORE_BARRIER_ENABLED=true \
  docker compose -f compose.yaml -f compose.debug.yaml --env-file .env \
  up -d --force-recreate haeon-card
sleep 20

LAB_PROFILE=before BEFORE_BARRIER_ENABLED=true CARD03_SCENARIO_PROFILE=demo \
  RUN_ID=$RUN bash tools/card03-concurrency.sh
```

기대 — 6,000,000원 두 건이 둘 다 **APPROVED**.

DB로 증명:

```
docker exec -i bookwave-haeon-lab-haeon-card-mysql-1 \
  sh -c 'mysql -uroot -proot_lab_only -t haeon_card -e "
    select l.limit_amount, l.used_amount, l.version
    from card_limits l join cards c on c.card_id = l.card_id
    where c.card_token = \"card-token-lab-001\";
    select correlation_id, status, read_used_amount, read_limit_version
    from authorization_requests order by auth_id;
    select count(*) tx, ifnull(sum(amount),0) tx_sum from card_transactions"'
```

기대 — used 12,000,000 > 한도 10,000,000, version 2, 두 건 모두 read_used=0 / v0, 거래 2건.

참지:

```
mkdir -p evidence/runs/$RUN
docker compose -f compose.yaml -f compose.debug.yaml --env-file .env \
  logs --no-color --tail=500 haeon-card > evidence/runs/$RUN/services.log

SERVICES_LOG=evidence/runs/$RUN/services.log \
CORRELATION_IDS=$RUN-A,$RUN-B \
BASELINE_USED_AMOUNT=0 EXPECTED_SERVICES=haeon-card \
RUN_ID=DETECT-$RUN bash tools/card03-detection-check.sh
```

기대 — **ALERT** (alerts=2, failures=0).

화면으로도 보여준다 (이 단계의 핵심 장면)

터미널 숫자는 개발자에게 와닿고, **회원 화면은 누구에게나 와닿는다**. 브라우저로 `http://localhost:8085/mypage` 를 열고 **새로고침** 을 누른다.

```
해온 플러스  ****  0001
  이용금액 12,000,000원 · 잔여한도 0원 / 이용한도 10,000,000원
  사용률 막대 100% (꼭 참)
```

최근 이용내역

북웨이브	6,000,000원	결제 승인	← 같은 시각, 승인번호만 다름
북웨이브	6,000,000원	결제 승인	

말할 문장: "회원 입장에서는 이렇게 됩니다. 한도가 1천만원인 카드인데 1천2백만원이 나갔고, 잔여한도가 0원입니다. 600만원 결제 두 건이 **둘 다 승인**으로 남아 있습니다."

두 건 승인이 안 나오면 재현 실패다. 촬영 중이면 **중단**하고 프로파일(`before`), 배리어(`true`), D-2 기준선부터 다시 본다.

D-4. After — 통제 확인

```
docker exec -i bookwave-haeon-lab-haeon-card-mysql-1 \
  sh -c 'mysql -uroot -proot_lab_only haeon_card' \
  < db/haeon-card/fixtures/004_card03_demo_reset.sql

TS=$(date -u +%Y%m%dT%H%M%S); RUN=HC03-AFTER-$TS; echo $RUN

LAB_PROFILE=after BEFORE_BARRIER_ENABLED=false \
  docker compose -f compose.yaml -f compose.debug.yaml --env-file .env \
  up -d --force-recreate haeon-card
sleep 20

LAB_PROFILE=after BEFORE_BARRIER_ENABLED=false CARD03_SCENARIO_PROFILE=demo \
  RUN_ID=$RUN bash tools/card03-concurrency.sh
```

기대 — 둘 중 한 건만 **APPROVED** , 나머지는 **DECLINED** / **LIMIT_EXCEEDED** .

A가 승인될지 B가 승인될지는 매번 다르다. 동시 요청이라 먼저 잠금을 잡는 쪽이 이긴다. 어느 쪽이든 정상이다. 중요한 건 "한 건만 승인"이라는 사실이다.

D-3과 같은 쿼리로 확인 — `used 6,000,000 ≤ 한도` , version 1, 거래 1건. 승인된 건은 `read_used=0 / v0` , 거절된 건은 `read_used=6,000,000 / v1` .

탐지도 D-3과 같은 명령 → 기대 **PASS** (9개 검사 전부).

화면으로도 보여준다

같은 마이페이지를 새로고침 한다. **Before**와 대비되는 장면이 화면만으로 완결된다.

```
해온 플러스  **** 0001
  이용금액 6,000,000원 · 잔여한도 4,000,000원 / 이용한도 10,000,000원

최근 이용내역
  북웨이브  6,000,000원  승인 거절 · 이용한도 초과  ← 막힌 쪽
  북웨이브  6,000,000원  결제 승인                  ← 통과한 쪽
```

말할 문장: "같은 공격을 그대로 다시 보냈는데, 이번엔 한 건만 승인되고 나머지는 한도 초과로 거절됐습니다. 잔여한도도 400만원으로 정상입니다."

	Before 화면	After 화면
이용금액	12,000,000원	6,000,000원
잔여한도	0원	4,000,000원
이용내역	승인 2건	승인 1건 · 거절 1건

D-5. 판정

	응답	used_amount	version	거래	탐지
Before	승인 2건	12,000,000 (한도 초과)	2	2건	ALERT
After	승인 1 / 거절 1	6,000,000	1	1건	PASS

이 표대로 나오면 합격. 하나라도 어긋나면 촬영을 멈춘다.

설명 한 줄: 차이는 코드 한 군데다 — 한도 행을 잠그고 최신 version으로 다시 판정하느냐.

	먼저 온 요청	나중 요청
Before	<code>read_used=0 / v0</code>	<code>read_used=0 / v0</code> ← 같은 값을 읽고 둘 다 승인
After	<code>read_used=0 / v0</code>	<code>read_used=6,000,000 / v1</code> ← 최신값으로 재판정 → 거절

E. 가상 웹셀 기반 결제서버 침입 재현 (핵심, 5분)

사회적으로 이슈가 된 카드사 침해의 초기 침입 → 데이터 접근 시도 → 탐지 → 차단·추가 검증 흐름을 카드 결제 서버 맥락에서 재현한다. 실행 가능한 웹셀·파일 업로드·명령 실행은 의도적으로 구현하지 않았다. 대신 전용 경로가 고정된 침입 징후(S1~S4)만 남기며, 결제 요청·Mock PG·카드사를 호출하지 않는다는 것까지 스스로 검증한다.

먼저 기본이 차단인 것을 보여준다.

```
curl -s -o /dev/null -w '%{http_code}\n' -X POST \
  http://localhost:8080/internal/v1/lab/incident-simulations/payment-server
# 기대: 403
```

명시적으로 켜고 실행한다.

```

INCIDENT_SIMULATION_ENABLED=true \
INCIDENT_SIMULATION_ACCESS_TOKEN=lab-incident-token-0918 \
  docker compose --env-file .env up -d --force-recreate bookwave-app
sleep 15

INCIDENT_SIMULATION_ACCESS_TOKEN=lab-incident-token-0918 \
RUN_ID=INCIDENT-SIM-$(date -u +%Y%m%dT%H%M%S) \
  bash tools/payment-server-incident-simulation.sh

```

기대 — `outcome=BLOCKED`, `mockPgCalled=false`, `haeonCardCalled=false`, 마지막 줄 `PASS`.

끝나면 반드시 되돌린다.

```
docker compose --env-file .env up -d --force-recreate bookwave-app
```

F. 네트워크 격리 점검 (선택, 2분)

```
bash tools/network-port-security-check.sh
```

DB 포트 미노출, 앱 포트 로컬호스트 바인딩, 카드사 포트 경계를 확인한다. 기대 바인딩은 `8080` / `8083` / `8085`. D의 디버그 프로파일이 올라가 있으면 `8084` 가 함께 잡힌다(둘 다 허용). **G(정리) 이후에 돌리는 것을 권한다.**

G. 정리 (필수)

1) 기준선 복원 + 마이페이지 이용내역 재시드

```
docker exec -i bookwave-haeon-lab-haeon-card-mysql-1 \
  sh -c 'mysql -uroot -proot_lab_only haeon_card' \
  < db/haeon-card/fixtures/003_card03_reset.sql
docker exec -i bookwave-haeon-lab-haeon-card-mysql-1 \
  sh -c 'mysql -uroot -proot_lab_only haeon_card' \
  < db/haeon-card/fixtures/007_portal_history_seed.sql
```

2) 정상 프로파일 + 내부 전용으로 복귀

```
docker compose -f compose.yaml -f compose.debug.yaml --env-file .env rm -sf haeon-card
docker network rm haeon_haeon_debug_net 2>/dev/null || true
LAB_PROFILE=normal BEFORE_BARRIER_ENABLED=false \
  docker compose --env-file .env up -d haeon-card
sleep 20
```

3) 포트 확인

```
printf '8084 차단(000 기대) : '; curl -s -o /dev/null -w '%{http_code}\n' --max-time 4 http://127.0.0.1:8084/actuator/health
printf '8085 개방(200 기대) : '; curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:8085/my
page
```

4) 상태

```
docker compose --env-file .env ps
```

기록 — 사용한 `RUN_ID` 들과 `git rev-parse --short HEAD` 를 발표 자료에 남긴다. 증거는 `evidence/runs/` 아래 `HC03-BEFORE-*` / `HC03-AFTER-*` / `DETECT-*` 폴더를 확인한다.

H. 막혔을 때

증상	원인	조치
<code>docker: command not found</code>	Windows 셸에서 실행	WSL 셸에서 다시
<code>card03-concurrency.sh</code> 연결 실패	8084 안 열림	D-1 디버그 프로파일 재생성
Before인데 한 건만 승인	프로파일·배리어 미적용 또는 기준선 미복원	<code>before / true</code> 로 재생성 후 D-2 다시
After인데 두 건 승인	after 재생성 누락	haeon-card 재생성 후 재실행
탐지가 <code>FAIL</code>	로그·DB 연결이 깨짐	촬영 중단. 로그 캡처 범위와 <code>CORRELATION_IDS</code> 확인
탐지가 즉시 중단	<code>CORRELATION_IDS / BASELINE_USED_AMOUNT</code> 누락	두 값은 필수
화면이 옛 버전	이미지 미재빌드 / 캐시	<code>build haeon-card bookwave-app</code> 후 Ctrl+F5
<code>/mypage</code> 가 404	옛 이미지	<code>docker compose --env-file .env build haeon-card</code>
로그인 401	포털 데이터 미적용	A-3 실행 후 다시
마이페이지 이용내역 0건	기준선 복원으로 승인 데이터 삭제	<code>007_portal_history_seed.sql</code> 재실행
주 카드가 해온 플러스로 보임	<code>006_portal_seed.sql</code> 최신본 미적용	006 다시 실행(재실행 안전)
D 이후 마이페이지 숫자가 B와 다름	D가 한도·내역을 바꿈	정상이다. 촬영 순서 규칙대로 B·C를 먼저

I. 마무리 한 문단

격리된 합성 결제 환경에서 정상 승인·역등·한도 거절을 먼저 보였다. 결제는 북웨이브에서 시작하고 결과는 해온카드에 기록되며, 회원은 해온카드 마이페이지에서 같은 승인번호로 확인한다. 그다음 동시 승인 경합으로 한도 이중 사용을 재현하고(Before: 12,000,000원 승인, 탐지 ALERT), 행 잠금과 최신 version 재판정을 적용해 차단되는 것(After: 6,000,000원, 탐지 PASS)을 로그·DB·자동 탐지 세 가지 증거로 동시에 증명했다.

부록 A. 리허설 기록 (2026-09-18)

전 단계를 실제로 돌린 결과다. 위 기대값은 모두 이때 실측한 값이다.

단계	결과
A	8개 컨테이너 healthy · 8080/8085//mypage 200 · 8085의 /internal 404 · 8084 외부 000
B	화면 점검 PASS (35/35) — 동작 19 + 구조 16(중복 id·앵커·aria-label·배경 밴드)
C	16,800원 승인 → 먹등 재조회 식별자 동일 → 마이페이지에 같은 승인번호, 잔여 100,000 → 83,200원. 한도 초과 200 DECLINED , 필수값 누락·상한 초과 400
D	Before ALERT (승인 2건·used 12,000,000) / After PASS (승인 1건·used 6,000,000)
F	호스트 포트 8건·네트워크 7건 전부 PASS
G	8084 000 / 8085 200 / 기준선 복원 확인
단위 테스트	haeon-card 12 · bookwave-app 12 · mock-pg 6 — 전부 통과

이번 리허설에서 확인한 것 세 가지

- After의 승인/거절 주체는 매번 바뀐다.** 이번엔 B가 승인되고 A가 거절됐다(직전 리허설은 반대). 동시 요청이라 정상이며, 판정 기준은 "둘 중 한 건만 승인"이다.
- 디버그 프로파일에서 8084가 열려 있어도 포트 경계는 유지된다.** 8084/portal/** 과 8085/internal/** 이 모두 404 였다.
- 점검 중 화면 폴더에 남아 있던 node_modules 가 이미지에 함께 담겨 8085 에서 서빙되고 있었다. 제거하고 .dockerignore · .gitignore 에 막아 두었다(정적 자산 21.3MB → 79KB).

부록 B. 해온카드 단독 시연 (북웨이브 없이)

카드사 쪽만 맡았거나 북웨이브 담당자의 준비가 아직이면, haeon-card 와 haeon-card-mysql 두 컨테이너만으로 시연의 대부분을 돌릴 수 있다. 아래 내용은 북웨이브 계열 컨테이너를 전부 내린 상태에서 실제로 돌려 확인한 것이다.

단독-0. 되는 것과 빠지는 것

런북 단계	단독 시연	비고
A. 준비·기준선	☒ 그대로	8080 확인만 건너뛰다
B-1. 북웨이브 서점	☒ 제외	가맹점 화면이 없다
B-2·B-3. 해온카드 화면·마이페이지	☒ 그대로	PASS (35/35) 확인
C. 결제 (북웨이브 화면)	☒ 대체	승인 API를 직접 호출해 같은 것을 보여준다 (B-3 아래)
D. CARD-03 Before/After	☒ 그대로	원래 북웨이브를 쓰지 않는다
E. 가상 웹셀 기반 결제서버 침입 재현	☒ 제외	북웨이브 쪽 기능
F. 네트워크 격리 점검	☒ 제외	8개 컨테이너 전체 기준이라 일부만 뜨면 FAIL
G. 정리	☒ 그대로	

핵심인 D단계는 영향이 없다. `card03-concurrency.sh` 는 호스트에서 카드 승인 API(8084)를 직접 호출하므로 북웨이브·결제대행사를 거치지 않는다.

단독-1. 기동과 기준선

```
docker compose --env-file .env up -d haeon-card      # mysql은 의존성으로 함께 뜬다
docker compose --env-file .env ps

docker exec -i bookwave-haeon-lab-haeon-card-mysql-1 \
  sh -c 'mysql -uroot -proot_lab_only haeon_card' \
  < db/haeon-card/fixtures/003_card03_reset.sql
docker exec -i bookwave-haeon-lab-haeon-card-mysql-1 \
  sh -c 'mysql -uroot -proot_lab_only haeon_card' \
  < db/haeon-card/fixtures/007_portal_history_seed.sql
```

기대 — `haeon-card` , `haeon-card-mysql` 두 개가 `healthy` . 본문 A-4 표와 같은 네 줄.

단독-2. 화면 (본문 B-2·B-3 그대로)

```
bash tools/portal-ui-check.sh      # 기대: PASS (35/35)
```

브라우저로 `localhost:8085` → 상단 MY → 로그인 → 마이페이지. 본문과 완전히 같다.

단독-3. 승인 — 결제 화면 대신 승인 API로 (본문 C 대체)

카드사 승인 API는 내부망 전용이라 먼저 디버그 포트를 연다.

```
docker compose -f compose.yaml -f compose.debug.yaml --env-file .env \
  up -d --force-recreate haeon-card
sleep 20
```

승인과 먹등 재시도를 한 번에 보여준다.

```
bash tools/haeon-card-smoke.sh
```

기대 — `first-response.json` 과 `replay-response.json` 의 `authorizationNo` 가 같다. 증거는 `evidence/runs/HC-SMOKE-*/` 에 남는다.

그 다음 **마이페이지를 새로고침**한다. 방금 승인된 건이 같은 승인번호로 이용내역 맨 위에 올라오고, 해온 플러스 ******0001**의 이용금액·잔여한도가 그만큼 바뀐다.

말할 문장: "가맹점 화면 대신 승인 API를 직접 호출했을 뿐, 카드사가 하는 일은 똑같다. 승인 기록이 카드사 DB에 남고 회원은 마이페이지에서 그걸 본다."

계약 경계까지 보여주려면:

```
HAEON_CARD_URL=http://localhost:8084 bash tools/card-api-contract-check.sh
```

승인·먹등 재시도·한도 초과·먹등키 충돌·가맹점 인증 실패를 한 번에 확인한다.

단독-4. CARD-03 (본문 D 그대로)

본문 **D-2 ~ D-5**를 그대로 실행한다. 바꿀 것이 없다. D-1(디버그 포트 열기)은 단독-3에서 이미 했다.

단독-5. 정리 (본문 G 그대로)

```
docker exec -i bookwave-haeon-lab-haeon-card-mysql-1 \
  sh -c 'mysql -uroot -proot_lab_only haeon_card' \
  < db/haeon-card/fixtures/003_card03_reset.sql
docker exec -i bookwave-haeon-lab-haeon-card-mysql-1 \
  sh -c 'mysql -uroot -proot_lab_only haeon_card' \
  < db/haeon-card/fixtures/007_portal_history_seed.sql

docker compose -f compose.yaml -f compose.debug.yaml --env-file .env rm -sf haeon-card
docker network rm haeon_haeon_debug_net 2>/dev/null || true
LAB_PROFILE=normal BEFORE_BARRIER_ENABLED=false \
  docker compose --env-file .env up -d haeon-card
```

전체 환경으로 되돌리려면 `docker compose --env-file .env up -d` 한 줄이면 된다.

단독-6. 나올 수 있는 질문

"북웨이브 서버가 꺼져 있는데 왜 이용내역에 '북웨이브'가 뜨나요?"

가맹점 이름은 카드사 DB(`merchants` 테이블)가 갖고 있는 값이다. 가맹점 서버가 살아 있는지와 무관하다. 오히려 승인 기록이 카드사 쪽에 독립적으로 남는다는 것을 보여주는 장면이다.

"그럼 결제는 누가 시작하나요?"

평소에는 북웨이브다. 지금은 그 자리를 승인 API 직접 호출로 대신했을 뿐이고, 카드사가 판정하는 방식은 완전히 같다. 승인 여부는 언제나 카드사가 정한다.

단독-7. 단독 실행 실측 (2026-09-18)

북웨이브 계열 6개 컨테이너를 모두 내린 상태에서 확인한 결과다.

항목	결과
마이페이지 화면	PASS (35/35)
승인 + 먹등	10,000원 APPROVED , 재시도 승인번호 동일
마이페이지 반영	같은 승인번호로 표시, 잔여한도 89,000원으로 감소
카드 API 계약	통과
CARD-03 Before	승인 2건 · used 12,000,000 · 탐지 ALERT
CARD-03 After	승인 1 / 거절 1 · used 6,000,000 · 탐지 PASS

부록 C. 촬영 가이드

촬영-1. 장비 구성 — 노트북 한 대에 다 올린다

팀원 둘이 각자 다른 파트를 맡았더라도 본 촬영은 한 대에서 한다.

이유 셋.

- 격리 자체가 시연 소재다.** `haeon_card_net` · `bookwave_db_net` · `lab_audit_net` 이 `internal: true` 고 호스트 포트는 전부 `127.0.0.1` 에만 묶여 있다. 두 대로 쪼개면 이걸 다 풀어야 하고, 그러면 F단계와 D-1의 "8084는 평소 내부 전용" 설명이 성립하지 않는다.
- 서비스 간 주소가 Docker DNS 이름이다.** `bookwave-app` → `http://mock-pg:8083` , `mock-pg` → `http://haeon-card:8084` . 쪼개면 상대 IP로 바뀌어야 하고, IP는 WiFi를 옮길 때마다 달라진다. 촬영 당일 변수로 들이기엔 나쁜 종류다.
- correlationId로 세 서비스 로그를 잇는 장면이 한 대에서만 자연스럽다.**

두 대로 쪼개려면 `MOCK_PG_BASE_URL · HAEON_CARD_BASE_URL` 을 상대 IP로, 포트 바인딩을 `0.0.0.0: 8084`를 네트워크에 노출해야 하고, `mock-pg` 를 어느 쪽에 둘지도 정해야 한다. `network-port-security-check.sh` 는 FAIL이 난다. 얻는 것에 비해 잃는 게 크다.

따로 찍고 싶다면 쪼개지 말고 복제한다. 각자 노트북에 `git clone` 후 `docker compose --env-file .env up -d` 하면 전체 스택이 독립적으로 돈다. 카드사 담당은 부록 B(단독 시연)를 그대로 쓰면 된다.

단 한 장면만 예외다. C-1의 "북웨이브에서 결제 → 해온카드 마이페이지에 같은 승인번호"는 같은 DB를 봐야 성립한다. 두 영상을 편집으로 붙이면 승인번호가 맞지 않는다. **이 장면만큼은 한 머신에서 연속으로 찍는다.**

촬영-2. 화면 배치

단계	배치
B · C	브라우저만 크게. 창 두 개(<code>8080</code> 북웨이브 / <code>8085/mypage</code> 해온카드)
D	왼쪽 터미널 / 오른쪽 브라우저(마이페이지)
F · G	터미널만

B·C에서는 터미널을 아예 접어둔다. **D에서만 터미널을 꺼내면 화면 전환 자체가 "이제부터 내부를 들여다본다"는 신호가 된다.**

촬영-3. 터미널이 꼭 필요한 구간

터미널에서 한 작업이 화면에 반영되는지는 구간마다 다르다. 실측 결과다.

터미널 작업	화면에 보이냐	진행 방식
CARD-03 Before 실행	☒ 그대로 보임	터미널 → 화면 전환해서 설명
CARD-03 After 실행	☒ 그대로 보임	터미널 → 화면 전환해서 설명
승인 API 직접 호출(<code>haeon-card-smoke.sh</code>)	☒ 보임	터미널 → 화면 전환
DB 쿼리(<code>read_used_amount / read_limit_version</code>)	☒ 안 보임	터미널에서 읽고 설명
탐지 스크립트(<code>ALERT / PASS</code>)	☒ 안 보임	터미널에서 읽고 설명
네트워크 격리 점검	☒ 안 보임	터미널 전용

화면에 없는 두 가지는 **일부러 뺐** 것이다. `version` 은 CARD-03 증거·운영 확인용이라 회원 응답에 넣지 않기로 했고, 탐지 판정은 DB·로그를 대조하는 증이라 회원 화면과 다르다.

화면은 "무슨 일이 일어났나", 터미널은 "왜 일어났나". 청중이 터미널 메시지를 해석할 필요는 없다. 결과는 화면으로 보여주고, 터미널은 발표자가 읽어주는 근거로 쓴다.

촬영-4. D단계 한 사이클 진행 순서

1. **터미널** — `card03-concurrency.sh` 실행 → 응답 두 건 다 **APPROVED** (3초)
2. **브라우저로 전환** — 마이페이지 새로고침 → "회원 입장에서 이렇게 됩니다. 한도 1천만원인데 1천2백만원이 나갔고 잔여한도가 0원입니다."
3. **터미널로 복귀** — DB 쿼리 → "왜 이렇게 됐냐면, 두 요청이 같은 값(`read_used=0 / v0`)을 읽었습니다."
4. **터미널** — 탐지 → **ALERT**
5. **After 반복** — 2번 화면이 정상으로 바뀌는 것으로 마무리

촬영-5. 터미널을 보기 좋게

출력을 지어낸 가짜 터미널은 쓰지 않는다. 로그·DB·자동 탐지로 증명한다는 발표에서 터미널을 연출하면 질의응답과 평가 모두에서 증거 신뢰를 잃는다. 아래처럼 실제 출력을 읽기 좋게 만든다.

방법	어떻게
터미널 꾸미기	폰트 18~20pt, 밝은 테마, <code>PS1='\$ '</code> 로 프롬프트 단순화
요약만 띄우기	<code>cat evidence/runs/DETECT-.../detection/summary.txt grep -E '^(PASS ALERT)'</code>
미리 실행해 두기	D단계는 컨테이너 재생성에 20초씩 걸린다. 미리 돌려 <code>evidence/runs/</code> 를 만들어 두고 결과를 열되, "미리 돌린 결과입니다"라고 밝힌다
증거 리포트 화면	<code>bash tools/evidence-report.sh</code> → <code>evidence/report.html</code> 를 브라우저로 연다 (실제 산출물만 읽으므로 정직)

촬영-6. 증거 리포트 화면

```
bash tools/evidence-report.sh          # -> evidence/report.html
```

`evidence/runs/` 의 최신 실행을 읽어 **판정 한 장**을 만든다. 브라우저로 열면 된다(서버 불필요).

- 판정 요약 7칸 — 가상 웹шел 기반 침입 징후 · CARD-03 Before/After · 회원 화면 점검 · 네트워크 격리 · 결제 왕복 · 카드 스모크
- 가상 웹шел 기반 침입 재현 — S3 **ALERT** , S4 차단·추가 검증 **PASS** , Mock PG·카드사 미호출
- CARD-03 Before/After 나란히 — 응답·사용액·`read_used_amount / read_limit_version` ·탐지 검사 목록
- 회원 화면 점검 항목 전체와 캡처 4장
- 호스트 포트·네트워크 표

값은 전부 실행 산출물에서 읽으며, 촬영 직전에 **매번 새로 만든다**. 마지막에 이 화면을 띄우고 "지금 보신 것들이 전부 여기 남아 있습니다"로 마무리한다.